Doble Grado Ingeniería Informática y Administración y Dirección de Empresas

Sistemas Concurrentes y Distribuidos

Práctica - Relaciones de Ejercicios

Ismael Sallami Moreno

25 de octubre de 2024

Índice

I Relación de Ejercicios del Tema 1 **3**

1. Ejercicio 1 **3**
1.1. Solución . . . 3
1.2. Análisis . . . 3
1.3. Escenarios posibles . . . 3
1.3.1. Posibles secuencias de ejecución . . . 3
1.3.2. Valores posibles para `x` . . . 4
1.4. Conclusión . . . 4

2. Ejercicio 2 **4**

3. Ejercicio 10 **4**
3.1. Problema de Lógica de Programas Resuelto . . . 5
3.2. Resolución 12 . . . 6

4. Ejercicio 14 **7**
4.1. Resolución 14 . . . 7

5. Ejercicio 15 **7**
5.1. Resolucion 15 . . . 8

6. Ejercicio 22 **9**
6.1. Estudio de Valores Finales . . . 9
6.2. Solución y Explicación Detallada . . . 10
6.3. Resultado Final . . . 10

7. Pregunta 24 **11**
7.1. Resolución 24 . . . 11

8. Ejercicio 25 **11**
8.1. Resolución 25 . . . 11
8.2. Resolución 25 . . . 11

9. Ejercicio 26 **12**
9.1. Resolución 26 . . . 12

10. Ejercicio 28 **13**
10.0.1. Resolucion del Ejercicio 28 . . . 13

11. Ejercicio 33 **13**
11.1. Resolución 33 . . . 13
11.1.1. Invariante del bucle . . . 13
11.1.2. Inicialización . . . 13
11.1.3. Mantenimiento . . . 14
11.1.4. Terminación . . . 14

12. Ejercicio 36 **14**
12.1. Resolución 36 . . . 15
12.1.1. Inicialización . . . 15
12.1.2. Mantenimiento . . . 15
12.1.3. Terminación . . . 15

13. Ejercicio 38 **16**
13.1. Resolución 38 16
13.1.1. Inicialización 16
13.1.2. Invariante del bucle 16
13.1.3. Condición de finalización 16
13.1.4. Función de Decremento 16
13.1.5. Notas de clase 17

14. Problema 48 **17**
14.1. Explicación paso a paso 17

Parte I
Relación de Ejercicios del Tema 1

1. Ejercicio 1

1.Considerar el siguiente fragmento de programa para 2 procesos P1 y P2 Los dos procesos pueden ejecutarse a cualquier velocidad. ¿Cuáles son los posibles valores resultantes para la variable `x`? Suponer que `x` debe ser cargada en un registro para incrementarse y que cada proceso usa un registro diferente para realizar el incremento.

```
{ variables compartidas }

var x : integer := 0 ;
Process P1;                     Process P2;
    var i: integer;                     var j: integer;
begin                           begin
    for i:= 1 to 2 do begin             for i:= 1 to 2 do begin
        x:= x + 1;                              x:= x + 1;
    end                                 end
end
```

1.1. Solución

El problema en la imagen muestra dos procesos, **P1** y **P2**, que están ejecutando un ciclo que incrementa la variable compartida `x` en 1, dos veces cada uno. Ambos procesos pueden ejecutarse a cualquier velocidad, y la variable compartida `x` se debe cargar en un registro antes de ser incrementada. Además, se especifica que cada proceso usa un registro diferente para realizar el incremento, lo cual indica que no hay sincronización inmediata entre los incrementos de los dos procesos.

1.2. Análisis

El incremento `x := x + 1` no es una operación atómica, lo que significa que:

1. El proceso carga el valor de `x` desde la memoria en su propio registro.
2. Luego le suma 1 a ese valor en el registro.
3. Finalmente almacena el valor de vuelta en la variable `x`.

Dado que los procesos pueden ejecutarse a diferentes velocidades, puede ocurrir una **condición de carrera**. Esto significa que los resultados dependen del orden en el que se ejecuten las instrucciones de los dos procesos.

1.3. Escenarios posibles

El valor inicial de `x` es 0, y cada proceso realiza dos incrementos, por lo que hay cuatro incrementos en total. El valor máximo de `x` debe ser 4, pero debido a la falta de atomicidad en el incremento, podrían producirse otros resultados.

1.3.1. Posibles secuencias de ejecución

1. **Ejecución sin interferencia (ideal):**
   - P1 realiza ambos incrementos.
   - P2 realiza ambos incrementos.
   - Resultado: $x = 4$.
2. **Intercalado mínimo:**
   - P1 realiza un incremento.

- P2 realiza un incremento.
- P1 realiza el segundo incremento.
- P2 realiza el segundo incremento.
- Resultado: $x = 4$.

3. **Condición de carrera (interferencias):**
   - P1 y P2 leen el valor de `x` (por ejemplo, ambos leen `x = 0`).
   - Ambos suman 1 en sus respectivos registros.
   - Ambos escriben el valor de vuelta en `x`, pero el valor final de `x` es solo 1.
   - Esto puede ocurrir varias veces, lo que lleva a resultados incorrectos.

1.3.2. Valores posibles para `x`

- En el mejor de los casos, todos los incrementos se realizan correctamente y `x = 4`.
- En el peor de los casos, debido a la interferencia entre los procesos, algunos incrementos pueden sobrescribir los resultados de otros, lo que podría llevar a un valor de `x` de **2** o **3**, si varios incrementos fallan en contabilizarse.

1.4. Conclusión

Los posibles valores de `x` al final de la ejecución de ambos procesos son **2**, **3** o **4**, dependiendo de la intercalación de las instrucciones entre los procesos P1 y P2 debido a la condición de carrera.

2. Ejercicio 2

¿Cómo se podría hacer la copia del fichero `f` en otro `g`, de forma concurrente, utilizando la instrucción concurrente `cobegin-coend`? Para ello, suponer que:

- Los archivos son una secuencia de ítems de un tipo arbitrario `T`, y se encuentran ya abiertos para lectura (`f`) y escritura (`g`). Para leer un ítem de `f` se usa la llamada a función `leer(f)` y para saber si se han leído todos los ítems de `f`, se puede usar la llamada `fin(f)` que devuelve verdadero si ha habido al menos un intento de leer cuando ya no quedan datos. Para escribir un dato `x` en `g` se puede usar la llamada a procedimiento `escribir(g,x)`.
- El orden de los ítems escritos en `g` debe coincidir con el de `f`.
- Dos accesos a dos archivos distintos pueden solaparse en el tiempo.

```
cobegin
    while not fin(f) do
        x := leer(f);
        escribir(g, x);
coend
```

3. Ejercicio 10

Para los siguientes fragmentos de código, obtener la poscondición adecuada para convertirlo en un triple demostrable con la Lógica de Programas:

- (a) {i < 10} i = 2 * i + 1 {}
- (b) {i > 0} i = i - 1; {}
- (c) {i > j} i = i + 1; j = j + 1 {}
- (d) {falso} a = a + 7; {}
- (e) {verdad} i = 3; j = 2 * i {}
- (f) {verdad} c = a + b; c = c / 2 {}

3.1. Problema de Lógica de Programas Resuelto

Para los siguientes fragmentos de código, obtener la poscondición adecuada para convertirlo en un triple demostrable con la Lógica de Programas:

- (a) {i < 10} i = 2 * i + 1 {i < 21}
  - **Precondición:** $i < 10$
  - **Operación:** $i = 2 * i + 1$
  - **Poscondición:** Como inicialmente $i < 10$, al duplicar i y sumar 1, el resultado máximo es $2 * 9 + 1 = 19$, luego $i < 21$.

  Para el cálculo del mismo debemos de calcular la nueva i para la poscondición, $\frac{i-1}{2} < i$. La i de la derecha es la i nueva, asi que la despejamos y nos queda que $i < 21$. Y así con los demás.
- (b) $\{i > 0\}\ i = i - 1\ \{i \geq 0\}$
  - **Precondición:** $i > 0$
  - **Operación:** $i = i - 1$
  - **Poscondición:** Restando 1 a i que es mayor a 0, el nuevo valor de i será mayor o igual a 0.
- (c) $\{i > j\}\ i = i + 1;\ j = j + 1\ \{i > j\}$
  - **Precondición:** $i > j$
  - **Operación:** $i = i + 1;\ j = j + 1$
  - **Poscondición:** Sumando 1 tanto a i como a j no cambia la desigualdad inicial, luego $i > j$.
- (d) {falso} $a = a + 7;$ {verdad}
  - **Precondición:** falso
  - **Operación:** $a = a + 7$
  - **Poscondición:** Cualquier acción derivada de una precondición falsa puede tener cualquier poscondición, por eso obtenemos verdad.
- (e) {verdad} $i = 3;\ j = 2 * i\ \{j = 6\}$
  - **Precondición:** verdad
  - **Operación:** $i = 3;\ j = 2 * i$
  - **Poscondición:** Dado que i se establece en 3, y luego $j = 2 * 3$, entonces $j = 6$.
- (f) {verdad} $c = a + b;\ c = c/2\ \{c = \frac{a+b}{2}\}$
  - **Precondición:** verdad
  - **Operación:** $c = a + b;\ c = c/2$
  - **Poscondición:** Dado que c se establece como la suma de a y b, y luego se divide entre 2, $c = \frac{a+b}{2}$.

Ejercicio 12

Si el triple $\{P\}\ C\ \{Q\}$ es demostrable, indicar por qué los siguientes triples también lo son (o no se pueden demostrar y por qué):

(a) $\{P\}\ C\ \{Q \vee P\}$

(b) $\{P \wedge D\}\ C\ \{Q\}$

(c) $\{P \vee D\}\ C\ \{Q\}$

(d) $\{P\}\ C\ \{Q \vee D\}$

(e) $\{P\}\ C\ \{Q \wedge P\}$

Debemos de tener en cuenta estas dos fórminulas para poder resolver el ejercicio:

- Regla de la consecuencia 1

$$\frac{\{P\}\ S\ \{Q\},\ \{Q\} \to \{R\}}{\{P\}\ S\ \{R\}} \tag{1}$$

- Regla de la consecuencia 2

$$\frac{\{R\} \to \{P\},\ \{P\}\ S\ \{Q\}}{\{R\}\ S\ \{Q\}} \tag{2}$$

La lógica que vamos a seguir para resolver este problema es que si hay una disyunción en la poscondición, entonces la poscondición es más débil que la original, por lo que el triple es demostrable. Si hay una conjunción en la poscondición, entonces la poscondición es más fuerte que la original, por lo que el triple no es demostrable. Ya que al ser posible que se de este caso o el otro, es más fácil de que se cumpla.

3.2. Resolución 12

Si el triple $\{P\}\ C\ \{Q\}$ es demostrable, indicar por qué los siguientes triples también lo son (o no se pueden demostrar y por qué):

(a) $\{P\}\ C\ \{Q \vee P\}$

- **Precondición:** P
- **Operación:** C
- **Postcondición:** $Q \vee P$
- **Explicación:** Dado que $Q \to (Q \vee P)$ es una implicación verdadera, la postcondición se debilita de Q a $Q \vee P$.

(b) $\{P \wedge D\}\ C\ \{Q\}$

- **Precondición:** $P \wedge D$
- **Operación:** C
- **Postcondición:** Q
- **Explicación:** La precondición se fortalece de P a $P \wedge D$, y dado que $P \wedge D \to P$ es una implicación verdadera, se mantiene la postcondición Q.

(c) $\{P \vee D\}\ C\ \{Q\}$

- **Precondición:** $P \vee D$
- **Operación:** C
- **Postcondición:** Q
- **Explicación:** No es demostrable. La precondición se debilita de P a $P \vee D$, lo cual no garantiza que Q sea verdadero después de C. D puede no estar relacionado con P o Q.

(d) $\{P\}\ C\ \{Q \vee D\}$

- **Precondición:** P
- **Operación:** C
- **Postcondición:** $Q \vee D$
- **Explicación:** Dado que $Q \to (Q \vee D)$ es una implicación verdadera, la postcondición se debilita de Q a $Q \vee D$.

(e) $\{P\}\ C\ \{Q \wedge P\}$

- **Precondición:** P
- **Operación:** C
- **Postcondición:** $Q \wedge P$
- **Explicación:** No es demostrable. La postcondición se fortalece de Q a $Q \wedge P$, lo cual no garantiza que P siga siendo verdadero después de C, especialmente si C modifica P.

4. Ejercicio 14

Dado el programa `int x = 5, y = 2; cobegin <x = x + y >;<y = x * y >coend;`, obtener:

(a) Valores finales de x e y

(b) Valores finales de x e y si quitamos los símbolos <>de instrucción atómica.

4.1. Resolución 14

Podemos tener los casos simples, es decir, que x=7 e y=14, o que x=15 e y=10. En el caso de que no sean atómicas, el resultado sería x=7 e y=10.

Dado el programa `int x = 5, y = 2; cobegin <x = x + y >;<y = x * y >coend;`, obtener:

(a) **Valores finales de x e y:**

- **Precondición:** $x = 5,\ y = 2$
- **Operación:** $<x = x + y>;\ <y = x * y>$
- **Postcondición:** Como las operaciones dentro de <><>son atómicas, se ejecutan sin interrupciones:
  - Si $x = x + y$ se ejecuta primero, $x = 5 + 2 = 7$. Luego $y = 7 * 2 = 14$.
  - Si $y = x * y$ se ejecuta primero, $y = 5 * 2 = 10$. Luego $x = 5 + 10 = 15$.

  En cualquier orden, el valor final de x e y será diferente. Así que no se puede determinar un único resultado final sin el orden específico de ejecución.

(b) **Valores finales de x e y sin las instrucciones atómicas:**

- **Precondición:** $x = 5,\ y = 2$
- **Operación:** *cobegin* $x = x + y\ ;\ y = x * y$ *coend*
- **Postcondición:** Si las operaciones no son atómicas, pueden interrumpirse entre sí, llevando a diferentes resultados basados en el orden de ejecución:
  - Si $x = x + y$ se ejecuta primero completamente, obtenemos $x = 5 + 2 = 7$. Luego $y = 5 * 2 = 10$. Entonces $x = 7$ y $y = 10$.
  - Si $y = x * y$ se ejecuta primero completamente, obtenemos $y = 5 * 2 = 10$. Luego $x = 5 + 10 = 15$. Entonces $x = 15$ y $y = 10$.

  Pero como no sabemos el orden de las operaciones, también podemos obtener $x = 7$ y $y = 14$. Así que el resultado final puede variar.

5. Ejercicio 15

15. Comprobar si la demostración del triple $\{x \geq 2\}$ $<x = x-2>;$ $\{x \geq 0\}$ interfiere con los teoremas siguientes:

(a) $\{x \geq 0\}$ $<x = x + 3>$ $\{x \geq 3\}$

(b) $\{x \geq 0\}$ $<x = x + 3>$ $\{x \geq 0\}$

(c) $\{x \geq 7\}$ $<x = x + 3>$ $\{x \geq 10\}$

(d) $\{y \geq 0\}$ $<y = y + 3>$ $\{y \geq 3\}$

(e) $\{x \text{ es impar}\}$ $<y = x + 1>$ $\{y \text{ es par}\}$

5.1. Resolucion 15

Advertencia

NO GARANTIZO QUE ESTE BIEN, PERO ES LO QUE HE ENTENDIDO DE LA TEORIA

Comprobar si la demostración del triple $\{x \geq 2\}$ $<x = x - 2>$ $\{x \geq 0\}$ interfiere con los teoremas siguientes:

(a) $\{x \geq 0\}$ $<x = x + 3>$ $\{x \geq 3\}$

- **Precondición:** $x \geq 0$
- **Operación:** $x = x + 3$
- **Postcondición:** $x \geq 3$
- **Explicación:**
  - La precondición indica que antes de ejecutar la instrucción, x es mayor o igual a 0.
  - La operación $x = x + 3$ incrementa el valor de x en 3 unidades.
  - Si x es mayor o igual a 0 antes de la operación, después de sumarle 3, x será mayor o igual a 3.
  - La implicación $Q \to (Q \vee P)$ es siempre verdadera, y dado que $Q = x \geq 3$ y $P = x \geq 0$, la postcondición se debilita de Q a $Q \vee P$.
- **Interferencia:** No hay interferencia porque debilitar la postcondición no afecta la validez del triple.

(b) $\{x \geq 0\}$ $<x = x + 3>$ $\{x \geq 0\}$

- **Precondición:** $x \geq 0$
- **Operación:** $x = x + 3$
- **Postcondición:** $x \geq 0$
- **Explicación:**
  - La precondición indica que antes de ejecutar la instrucción, x es mayor o igual a 0.
  - La operación $x = x + 3$ incrementa el valor de x en 3 unidades.
  - Si x es mayor o igual a 0 antes de la operación, seguirá siendo mayor o igual a 0 después de sumarle 3.
  - La precondición y la postcondición son las mismas.
- **Interferencia:** No hay interferencia porque no hay cambio en las condiciones iniciales ni finales.

(c) $\{x \geq 7\}$ $<x = x + 3>$ $\{x \geq 10\}$

- **Precondición:** $x \geq 7$
- **Operación:** $x = x + 3$
- **Postcondición:** $x \geq 10$
- **Explicación:**
  - La precondición indica que antes de ejecutar la instrucción, x es mayor o igual a 7.
  - La operación $x = x + 3$ incrementa el valor de x en 3 unidades.
  - Si x es mayor o igual a 7 antes de la operación, después de sumarle 3, x será mayor o igual a 10.
  - La precondición se fortalece de $x \geq 2$ a $x \geq 7$, lo que no afecta la postcondición.
- **Interferencia:** No hay interferencia porque fortalecer la precondición no afecta la validez del triple.

(d) $\{y \geq 0\}$ $<y = y + 3>$ $\{y \geq 3\}$

- **Precondición:** $y \geq 0$

- **Operación:** $y = y + 3$
- **Postcondición:** $y \geq 3$
- **Explicación:**
  - La precondición indica que antes de ejecutar la instrucción, y es mayor o igual a 0.
  - La operación $y = y + 3$ incrementa el valor de y en 3 unidades.
  - Si y es mayor o igual a 0 antes de la operación, después de sumarle 3, y será mayor o igual a 3.
  - Aunque se usa una variable diferente (y en lugar de x), el principio sigue siendo el mismo.
- **Interferencia:** No hay interferencia porque el análisis es independiente de la variable usada.

(e) $\{x$ es impar$\}$ $<y = x + 1>$ $\{y$ es par$\}$

- **Precondición:** x es impar
- **Operación:** $y = x + 1$
- **Postcondición:** y es par
- **Explicación:**
  - La precondición indica que antes de ejecutar la instrucción, x es un número impar.
  - La operación $y = x + 1$ incrementa el valor de x en 1 unidad, asignando el resultado a y.
  - Si x es impar, sumar 1 resultará en un número par.
  - La operación $y = x + 1$ garantiza que si x es impar, y será par.
- **Interferencia:** No hay interferencia porque el valor de x después de la instrucción no afecta la postcondición sobre y.

6. Ejercicio 22

Estudiar cuáles son los valores finales de las variables x e y en el siguiente programa secuencial. Insertar los asertos adecuados entre llaves, antes y después de cada sentencia, para poder obtener una traza de demostración del programa, que incluya en su último aserto los valores finales de las variables.

(a) `int x = C1;`

(b) `int y = C2;`

(c) `x = x + y;`

(d) `y = x * y;`

(e) `x = x - y;`

6.1. Estudio de Valores Finales

Estudiar cuáles son los valores finales de las variables x e y en el siguiente programa secuencial. Insertar los asertos adecuados entre llaves, antes y después de cada sentencia, para poder obtener una traza de demostración del programa, que incluya en su último aserto los valores finales de las variables.

(a) `int x = C1;`

(b) `int y = C2;`

(c) `x = x + y;`

(d) `y = x * y;`

(e) `x = x - y;`

6.2. Solución y Explicación Detallada

```
// {x = C1, y = C2}
int x = C1;
// {x = C1, y = C2}
int y = C2;
// {x = C1, y = C2}
x = x + y;
// {x = C1 + C2, y = C2}
y = x * y;
// {x = C1 + C2, y = (C1 + C2) * C2}
x = x - y;
// {x = C1 + C2 - (C1 + C2) * C2, y = (C1 + C2) * C2}
```

- **Inicialización de x:**
  - **Código:** `int x = C1;`
  - **Aserto:** // {x = C1, y = C2}
  - **Explicación:** Inicializamos x con $C1$. En este momento, x es igual a $C1$ y y no está inicializado.
- **Inicialización de y:**
  - **Código:** `int y = C2;`
  - **Aserto:** // {x = C1, y = C2}
  - **Explicación:** Inicializamos y con $C2$. Ahora, x es igual a $C1$ y y es igual a $C2$.
- **Primera Operación (Suma):**
  - **Código:** `x = x + y;`
  - **Aserto:** // {x = C1 + C2, y = C2}
  - **Explicación:** Sumamos y a x y guardamos el resultado en x. Después de esta operación, x es igual a $C1 + C2$ y y sigue siendo $C2$.
- **Segunda Operación (Multiplicación):**
  - **Código:** `y = x * y;`
  - **Aserto:** // {x = C1 + C2, y = (C1 + C2) * C2}
  - **Explicación:** Multiplicamos x por y y guardamos el resultado en y. Después de esta operación, y es igual a $(C1 + C2) * C2$ y x sigue siendo $C1 + C2$.
- **Tercera Operación (Resta):**
  - **Código:** `x = x - y;`
  - **Aserto:** // {x = C1 + C2 - (C1 + C2) * C2, y = (C1 + C2) * C2}
  - **Explicación:** Restamos y de x y guardamos el resultado en x. Después de esta operación, x es igual a $C1 + C2 - (C1 + C2) * C2$ y y sigue siendo $(C1 + C2) * C2$.

6.3. Resultado Final

Después de ejecutar todas las operaciones, los valores finales de las variables son:

- $x = C1 + C2 - (C1 + C2) * C2$
- $y = (C1 + C2) * C2$

7. Pregunta 24

Dada la siguiente construcción de composición concurrente P:

```
cobegin
<x = x - 1> ; <x = x + 1> || <y = y + 1> ; <y = y - 1>
coend;
```

Demostrar que se cumple la invariancia de {x==y}, es decir, que {x==y} P {x==y}; es un triple cierto.

7.1. Resolución 24

Sea x = y es invariante

- $\langle x = x - 1 \rangle \mid \langle x = x + 1 \rangle \parallel \langle y = y + 1 \rangle \mid \langle y = y - 1 \rangle$
- $\{x = y\}$
- $\langle x = x - 1 \rangle$
- $\{x + 1 = y\} \rightarrow \{x = y - 1\}$
- $\langle x = x + 1 \rangle$
- $\{x - 1 = y - 1\} \rightarrow \{x = y\}$
- $\{x = y\}$
- $\langle y = y + 1 \rangle$
- $\{x = y - 1\}$
- $\langle y = y + 1 \rangle$
- $\{x = (y + 1) - 1\} \rightarrow \{x = y\}$

8. Ejercicio 25

Usando la regla de la conjunción, demostrar que:

```
{i>2}i:=2*i {i>4}
```

8.1. Resolución 25

Regla de la conjunción:

$$\frac{\{P_1\}S\{Q_1\},\{P_2\}S\{Q_2\}}{\{P_1 \wedge P_2\}S\{Q_1 \wedge Q_2\}}$$

8.2. Resolución 25

En este caso:

$$P_1 : i > 2 Q_1 : i > 4 S : i := 2 * i$$

Primero verificamos la corrección de P_1 con respecto a Q_1:

$$\{i > 2\}\ i := 2 * i\ \{i > 4\}$$

Para demostrar esto, consideramos el estado después de la asignación:

$$i := 2 * i \implies i = 2 * i$$

Dado que $i > 2$, multiplicando por 2 obtenemos:

$$2 * i > 4$$

Por lo tanto, $i > 4$.

Verificamos también P_2 y Q_2 en este caso no necesitamos otro, ya que:

$$P_2 : i > 2Q_2 : i > 4$$

Es evidente que

$$i > 2 \Rightarrow i > 4$$

Por lo tanto, con la regla de la conjunción, demostramos:

$$\{i > 2\}\ i := 2 * i\ \{i > 4\}$$

9. Ejercicio 26

26. Se dan los siguientes triples de Hoare:
{j > 1} $i = i + 2$; $j = j + 3$; {j > 4}
{i > 2} $i = i + 2$; $j = j + 3$; {i > 4}
Demostrar que estos triples implican que {j > 1, i > 2} $i = i + 2$; $j = j + 3$; {j > 4, i > 4}. ¿Qué regla se debe utilizar para la demostración?

9.1. Resolución 26

Utilizaremos la **Regla de la Conjunción** para demostrar que

$$\{j > 1, i > 2\}\text{i = i + 2;j = j + 3;}\{j > 4, i > 4\}.$$

La regla de la conjunción se enuncia como:

$$\frac{\{P_1\}S\{Q_1\},\{P_2\}S\{Q_2\}}{\{P_1 \wedge P_2\}S\{Q_1 \wedge Q_2\}}$$

Aquí:

$$P_1 : j > 1Q_1 : j > 4P_2 : i > 2Q_2 : i > 4S : i := i + 2; j := j + 3$$

Primero verificamos la corrección de P_1 con respecto a Q_1:

$$\{j > 1\}\ i := i + 2; j := j + 3\ \{j > 4\}$$

Para demostrar esto, consideramos el estado después de la asignación:

$$j := j + 3 \implies j = j + 3$$

Dado que $j > 1$, sumando 3 obtenemos:

$$j + 3 > 4$$

Por lo tanto, $j > 4$.

Verificamos también P_2 con respecto a Q_2:

$$\{i > 2\}\ i := i + 2; j := j + 3\ \{i > 4\}$$

Consideramos el estado después de la asignación:

$$i := i + 2 \implies i = i + 2$$

Dado que $i > 2$, sumando 2 obtenemos:

$$i + 2 > 4$$

Por lo tanto, $i > 4$.

Con la regla de la conjunción, combinamos ambos resultados:

$$\frac{\{j > 1\}S\{j > 4\}, \{i > 2\}S\{i > 4\}}{\{j > 1 \wedge i > 2\}S\{j > 4 \wedge i > 4\}}$$

Finalmente, demostramos que:

$$\{j > 1, i > 2\}\ i := i + 2; j := j + 3\ \{j > 4, i > 4\}$$

10. Ejercicio 28

Demostrar que la siguiente sentencia tiene la poscondición $\{x \geq 0, x^2 = a^2\}$.
`if` $a > 0$ `then` $x := a$ `else` $x := -a$
$\{V\}$ `if` $a > 0$ `then` $x := a$ `else` $x := -a\{x \geq 0, x^2 = a^2\}$

10.0.1. Resolucion del Ejercicio 28

$$\{x^2 \leq B\}\ S_1\ \{x^2 \leq B\}, \quad \{x^2 \leq B\}\ S_2\ \{x^2 \leq B\}$$

$$\{x^2 \leq B\} \quad \text{if } (B) \text{ then } S_1 \text{ else } S_2 \text{ endif } \{x^2\}$$

Directa:

$$\{\forall x\ \{x^2 \leq B\}\ x := a\ \{x = a,\ a > 0\} \rightarrow \{x^2 = a^2\}$$

Inversa (Else):

$$\{\forall x\ \{x^2 \leq a\}\ x := -a\ \{x = -a,\ a \leq 0\} \rightarrow \{x^2 = a^2\}$$

$$\{V\}$$

if (a > 0) then x := a else x := -a ;

$$\{x^2 = a^2\},\ x \geq 0$$

11. Ejercicio 33

Demostrar la corrección parcial del siguiente fragmento de programa:

```
sum := 0; j := 1;
while(j != c) do
begin
    sum := sum + j; j := j + 1;
end
{sum = c * (c - 1)/2}
```

11.1. Resolución 33

Para demostrar la corrección parcial del programa, necesitamos probar que, al final del bucle, la suma es igual a $\frac{c \times (c-1)}{2}$.

11.1.1. Invariante del bucle

Elegimos el invariante del bucle como:

$$I : sum = \frac{j \times (j-1)}{2}$$

Este invariante se elige porque representa la suma de los primeros $j - 1$ números naturales. Es relevante aquí porque, al incrementar j en cada iteración del bucle, queremos acumular la suma de todos los números hasta $j - 1$.

11.1.2. Inicialización

Antes de que el bucle comience, tenemos:

$$sum := 0; \quad j := 1$$

Sustituyendo estos valores en el invariante, obtenemos:

$$I : 0 = \frac{1 \times (1-1)}{2} = 0$$

Esto es cierto, por lo que el invariante se cumple inicialmente.

11.1.3. Mantenimiento

Si el invariante es verdadero antes de una iteración del bucle, debe seguir siendo verdadero después de la iteración. Supongamos que el invariante es verdadero antes de una iteración:

$$sum = \frac{j \times (j-1)}{2}$$

Durante la iteración, se ejecutan las siguientes instrucciones:

$$sum := sum + j; \quad j := j + 1$$

Después de la ejecución de estas instrucciones, los valores se actualizan a:

$$sum = \frac{j \times (j-1)}{2} + j$$

$$j = j + 1$$

Sustituyendo el nuevo valor de j en el invariante, tenemos:

$$sum = \frac{(j-1) \times (j-2)}{2} + j$$

$$= \frac{(j^2 - 3j + 2) + 2j}{2}$$

$$= \frac{j^2 - j + 2}{2}$$

Ajustando correctamente:

$$= \frac{(j^2 - j) + 2}{2}$$

$$= \frac{j(j-1)}{2}$$

Esto demuestra que el invariante se mantiene.

11.1.4. Terminación

El bucle termina cuando $j = c$. En este punto, el invariante se convierte en:

$$sum = \frac{c \times (c-1)}{2}$$

12. Ejercicio 36

El siguiente fragmento de programa calcula $\sum_{i=1}^{n} i!$. Demostrar que es correcto con el invariante:

$$sum = \sum_{j=1}^{i-1} j! \wedge f = i!$$

```
i := 1; sum := 0; f := 1;
while i \neq n + 1 do
begin
    sum := sum + f;
    i := i + 1;
    f := f * i;
end
```

12.1. Resolución 36

12.1.1. Inicialización

Antes de que el bucle comience, tenemos:

$$i := 1; \quad \text{sum} := 0; \quad f := 1$$

Sustituyendo estos valores en el invariante, obtenemos:

$$\text{sum} = \sum_{j=1}^{1-1} j! \wedge f = 1!$$

Lo que se simplifica a:

$$\text{sum} = 0 \wedge f = 1$$

Esto es cierto, por lo que el invariante se cumple inicialmente.

12.1.2. Mantenimiento

Si el invariante es verdadero antes de una iteración del bucle, debe seguir siendo verdadero después de la iteración. Supongamos que el invariante es verdadero antes de una iteración:

$$\text{sum} = \sum_{j=1}^{i-1} j! \wedge f = i!$$

Durante la iteración, se ejecutan las siguientes instrucciones:

$$\text{sum} := \text{sum} + f$$
$$i := i + 1$$
$$f := f * i$$

Después de la ejecución de estas instrucciones, los valores se actualizan a:

$$\text{sum} = \sum_{j=1}^{i-1} j! + i!$$
$$i = i + 1$$
$$f = i * i! = (i + 1)!$$

Sustituyendo estos valores en el invariante, obtenemos:

$$\text{sum} = \sum_{j=1}^{i} j!$$
$$f = (i + 1)!$$

Esto demuestra que el invariante se mantiene.

12.1.3. Terminación

El bucle termina cuando $i = n + 1$. En este punto, el invariante se convierte en:

$$\text{sum} = \sum_{j=1}^{n} j!$$
$$f = (n + 1)!$$

Esto es precisamente lo que queríamos demostrar. Por lo tanto, el programa es correcto con respecto a la especificación dada.

13. Ejercicio 38

Demostrar que para $n > 0$ el siguiente fragmento de programa termina.

```
i := 1; f := 1;
while i != n do
begin
    i := i + 1;
    f := f * i;
end
```

13.1. Resolución 38

Para demostrar que el programa termina, debemos mostrar que el bucle `while` eventualmente se detendrá. Utilizaremos la técnica de la **Función de Decremento**, que es una función que decrece con cada iteración del bucle y está acotada inferiormente. La idea es encontrar una medida que decrezca en cada iteración y eventualmente llegue a un valor límite donde el bucle se detendrá.

13.1.1. Inicialización

Antes de que el bucle comience, los valores iniciales son:

$$i := 1; \quad f := 1$$

13.1.2. Invariante del bucle

Elegimos el siguiente invariante del bucle:

$$1 \leq i \leq n$$

Este invariante se elige porque i comienza en 1 y se incrementa en cada iteración hasta alcanzar n.

13.1.3. Condición de finalización

La condición de finalización del bucle es $i = n$. El bucle se ejecuta mientras $i \neq n$, por lo que una vez que $i = n$, el bucle termina.

13.1.4. Función de Decremento

La función de decremento que utilizaremos es:

$$T(i) = n - i$$

Esta función mide la distancia entre i y n. Con cada iteración del bucle, i se incrementa en 1, por lo que $T(i)$ decrece en 1.

Inicialización: Al inicio, $i = 1$, por lo tanto:

$$T(i) = n - 1$$

Esto es mayor o igual a 0 para $n > 0$.

Mantenimiento: Durante cada iteración del bucle, i se incrementa en 1:

$$i := i + 1$$

$$T(i) = n - (i + 1) = n - i - 1$$

Como $T(i)$ decrece en cada iteración y está acotada inferiormente por 0, eventualmente $T(i) = 0$.

Terminación: El bucle termina cuando $i = n$, en cuyo punto:

$$T(i) = n - n = 0$$

Así, hemos demostrado que el bucle se detiene para $n > 0$.

13.1.5. Notas de clase

- La única variable `variante` en la condición del bucle `while B do` es `i`.
- `i` solo toma valores en la sucesión {1, 2, 3 ..., n^4}
- Nada impide que la variable `i` = `n` (condición de parada)

14. Problema 48

Dados $n \geq 0$, $i \leq n$, demostrar que el siguiente segmento de programa evalúa $\frac{n!}{(i!*(n-i)!)}$ dado el invariante $\{i > k \vee afact = i!\} \wedge \{n - i > k \vee bfact = (n - i)!\}$:

```
k := 0; fact := 1;
while (k \neq n) do
begin
    k := k + 1; fact := fact * k;
    if (k \leq i)
        then afact := fact;
    if (k \leq n - i)
        then bfact := fact
end
bcof = \frac{fact}{(afact * bfact)}
```

14.1. Explicación paso a paso

Para resolver este problema, debemos demostrar que el segmento de programa dado evalúa $\frac{n!}{(i!*(n-i)!)}$ utilizando el invariante proporcionado. Aquí está el paso a paso:

- Inicialización:
  - Se inicializa k a 0 y $fact$ a 1.
  - El invariante inicial es $\{i > k \vee afact = i!\} \wedge \{n - i > k \vee bfact = (n - i)!\}$.
- Bucle While:
  - El bucle se ejecuta mientras $k \neq n$.
  - Dentro del bucle, se incrementa k en 1 y se actualiza $fact$ multiplicándolo por k.
- Condiciones If:
  - Si $k \leq i$, se asigna $fact$ a $afact$. Esto asegura que $afact$ sea igual a $i!$ cuando $k = i$.
  - Si $k \leq n - i$, se asigna $fact$ a $bfact$. Esto asegura que $bfact$ sea igual a $(n - i)!$ cuando $k = n - i$.
- Finalización del Bucle:
  - Cuando el bucle termina, $fact$ será igual a $n!$ porque se ha multiplicado por todos los números de 1 a n.
- Cálculo de *bcof*:
  - Finalmente, se calcula *bcof* como $\frac{fact}{(afact*bfact)}$.
  - Dado que $fact = n!$, $afact = i!$, y $bfact = (n - i)!$, el resultado es $\frac{n!}{(i!*(n-i)!)}$.

```
k := 0; fact := 1;
while (k \neq n) do
begin
    k := k + 1; fact := fact * k;
    if (k \leq i)
        then afact := fact;
```

```
    if (k \leq n - i)
        then bfact := fact
end
bcof = \frac{fact}{(afact * bfact)}
```

Este paso a paso demuestra que el segmento de programa dado evalúa correctamente $\frac{n!}{(i!*(n-i)!)}$ utilizando el invariante proporcionado.